

SUMMARY

Full-stack engineer with 7 years of experience. Designed, built, and operated a B2B multi-tenant SaaS from **zero to production** — an architecture that absorbs event-day spikes from a ~1K baseline DAU to **~25K concurrent users** with **zero downtime and no over-provisioning**, serving **10+ business customers** — while delivering 20–30 event platforms a year for national institutions.

I'm strongest at keeping systems reliable on a lean budget: pre-rendered read paths, CDN caching, PostgreSQL tuning, and pre-event load testing. Primary stack is TypeScript; I've also shipped production systems in Java and C# where the problem called for it. In a 3-person team I mentored a junior engineer through code review — and I operate what I ship, end to end. Having co-founded the startup, I'm used to weighing technical decisions against product and cost.

Multi-tenant SaaS

Spiky-traffic design

0 → 1 architecture

Performance & reliability ownership

Team leading & mentoring

AI-native workflow

EXPERIENCE

Plancy · Founding Engineer / Head of Engineering (co-founder)

May 2021 – Jun 2026

Owned architecture, development, infrastructure, and operations across Plancy's two product lines for the MICE (events & exhibitions) industry — a multi-tenant SaaS and 20–30 event web/registration platforms a year. Sole senior engineer in a 3-person team (a PM/designer and a junior engineer).

MICE All-in-One — B2B multi-tenant SaaS (mice.ai.kr) · Lead Engineer

Feb 2024 – Jun 2026

- **0 → 1 architecture** — Started from the insight that an event agency's customers are *events*, *not people*: modeled each event as the tenant unit, so the isolation boundary matched the domain model — simplifying permissions, data scoping, and cost attribution — and infrastructure cost scaled predictably with projects rather than end users. Built from PoC to production, now in daily use by **10+ agency customers**.
↳ **Takeaway** · Matching the tenant boundary to the domain's real unit keeps isolation, permissions, and cost on one axis — complexity stays flat as the system scales.
- **Spiky, event-driven load** — Absorbed event-day peaks of **~25K concurrent users** against a ~1K baseline DAU across 3–4 large government-backed events per year: pre-rendered and CDN-cached the read paths, tuned PostgreSQL indexes and connection pooling, and validated capacity with pre-event load tests. **Zero downtime, no over-provisioning**.
↳ **Takeaway** · When peaks are predictable, making the read path static beats scaling out — cheaper and more certain.
- **Bottleneck diagnosis & Rust migration** — Diagnosed bulk document generation timing out on serverless as the platform's worst bottleneck; moved that hot path alone to a dedicated Rust (Axum) service via a strangler-fig migration, chosen for the CPU-bound rendering workload and memory footprint. **Eliminated failures on large batch jobs** and instrumented monitoring to catch the next bottleneck.
↳ **Takeaway** · Replacing only the bottleneck path — not rewriting the system — gave the best risk-to-impact ratio for a small team.

- **Abstracting repeated work** — Turned per-event custom registration forms (previously built by hand each time) into a **JSON-Schema-driven no-code form builder** customers configure themselves; opened the product to integrations with a public REST API and scoped API keys (developer center).
- **Identity & tenancy** — Consolidated three separate login flows into one identity model with per-event tenant isolation (PostgreSQL RLS); shipped self-service domain provisioning — auto-deduplicated subdomains or customer-purchased domains configured automatically via the Cloudflare API.
- **Event-operations toolkit** — Real-time business matching (meeting requests, live chat, scheduling), QR on-site check-in with registration-to-on-site funnel analytics, and scheduled/recurring invoicing and email with delivery and open tracking.
- **Quality & release safety** — Automated tests on critical flows plus preview deployments and rollback procedures kept releases safe with a 2-engineer team; wired Sentry and PostHog into an internal admin console so non-technical leadership had a single view of platform health.
- **Collaboration & AI-native workflow** — Drove requirements definition with the PM and a customer-feedback-to-roadmap cycle; mentored the junior engineer through code review. Established an AI-agent-centric workflow (Cursor, Claude Code) that let a small team sustain output across two product lines, and prototyped a RAG-based proposal assistant to validate the product's AI direction.

Stack TypeScript, Next.js, React, Node.js, Supabase (PostgreSQL, RLS, Realtime), Cloudflare, Docker, Vercel, TipTap/YJS, Tauri, Sentry, PostHog, GitHub Actions

Event Web & Registration Platforms — national institutions & large events ·

Oct 2021 – Oct 2025

Technical Lead

- **Direct stakeholder collaboration** — Worked directly with stakeholders at national institutions (e.g., National Hangeul Museum, Sculpture City Seoul) to scope requirements and deliver end to end — websites and real-time registration platforms, from planning through production operations.
- **Templatizing repeated builds** — Delivered **20–30 events per year** (each ~2,000 DAU at peak registration) over 3–4 years; abstracted the recurring site and registration flows into reusable templates to cut build time per event.
↳ **Takeaway** · "Structure anything you've built three times" — the level of abstraction decides both the speed and the quality of repeated delivery.
- **Environment ownership** — Owned infrastructure and deployment, plus the team's internal dev environment, tooling, and workflow.

Stack Next.js, TypeScript, React, Node.js, Supabase/PostgreSQL, Vercel, Cloudflare, Docker

Client Engineering — enterprise SI & prototyping · Software Engineer / Tech Lead

Oct 2021 – Oct 2025

- **Stack adaptability** — Delivered 3–6 month projects for enterprise clients, adapting stack and architecture per project — from frontend-only apps to serverless and full-stack systems.
- **Solar-design review platform** — Built an Autodesk Revit add-in (C#/ .NET) to extract BIM data, designed a custom file format (.pim, a ZIP of structured JSON), and surfaced reports, charts, and national-standard compliance checks on the web (Next.js + Supabase) — a demanding desktop-CAD-to-web integration.
- **Public-sector lookup system** — Java/Spring on a CSAP-certified cloud (NHN Cloud), with PII encryption and access-control design.

Stack C#/ .NET, Autodesk Revit API, Next.js, TypeScript, Java, Spring Boot, NHN Cloud, Supabase

TheScentLab · Founding Engineer (co-founder)

Sep 2020 – May 2021

Led product and frontend, and built the server alongside a junior backend engineer I mentored.

- Built and launched a personalized fragrance recommendation app (React Native/Expo) on iOS and Android; designed the recommendation engine over a **~70K-item** dataset structured by scent notes — Elasticsearch weighted scoring combined with survey-based personalization.
- Designed and deployed the NestJS backend as containers on AWS ECS. **Co-filed a patent** for the recommendation process; secured **~KRW 50M** in government startup funding.

Stack React Native, Expo, TypeScript, NestJS, Elasticsearch, PostgreSQL, AWS (ECS, EC2, RDS, Lambda)

Supaja · Frontend Lead (contract)

Apr 2020 – Sep 2020

- Designed and built the service's entire frontend solo; onboarded one developer new to React and left structure and docs so the service ran without me.

Weplanet · Web Developer Intern

Mar 2019 – Aug 2019

- Web prototyping across client projects — React frontends with Node (Express) and MySQL backends.

PERSONAL PROJECT

Bearings (thebearings.app) · Solo builder & operator — alongside job search

Jun 2026 – Present

"Which economic regime is your portfolio betting on — and which one quietly breaks it?" A regime read for what you already hold. Started as Cohort, a Korean investing-pace companion; pivoted to the global-facing Bearings. Solo-designed, shipped, and operated. (github.com/rayleighko/thebearings.app)

- Designed and shipped a PWA on Next.js 16 (App Router), React 19, and TypeScript; operate the macro-data pipeline (ECOS, FRED), observability (PostHog, Sentry), and deployment (Vercel) as a team of one.
- **Privacy-first by design** — no login, no brokerage linking, nothing stored; the regime read works without an account, removing friction for global community distribution.
- **AI safety boundary design** — Claude-powered briefing and chat behind a 3-layer safety filter that enforces, at the system level, the line between decision support and investment advice.
- **Product pivot & docs-first engineering** — architecture docs, roadmap, and a work journal maintained alongside the code; revalidated positioning after v1 and pivoted to a global regime tool, writing and executing the English-community launch plan solo.

Stack Next.js 16, React 19, TypeScript, Supabase, Claude API, Tailwind, PWA, PostHog, Sentry, Vercel

SKILLS

Fluent	TypeScript, React, Next.js, Node.js, NestJS, PostgreSQL, Supabase
Worked with	Java/Spring, C#/.NET, React Native, Tauri, Python, Express, MySQL
Infrastructure	AWS (ECS, EC2, RDS, Lambda), Docker, Vercel, Cloudflare, NHN Cloud, CI/CD, Sentry, PostHog
Areas	Multi-tenant SaaS, real-time systems, performance optimization, 0-to-1 product, AI-assisted development

EDUCATION · ADDITIONAL

- **Sahmyook University** — B.S. in Computer Science (Software major), Feb 2021
- **Military service** — Republic of Korea Marine Corps, IT operations (Feb 2014 – Nov 2015)
- **Patent** — Co-filed, personalized recommendation process (TheScentLab)
- **Community** — AUSG / AWSKRUG (AWS Korea user group), 2019
- **Languages** — Korean (native) · English (professional reading/writing; conversational speaking, improving)